



北方民族大学

本科毕业论文（设计）

题目：

基于 Vulkan 的实时光线追踪混合渲染器实现

学院名称： 计算机科学与工程学院

专业： 软件工程

学生姓名： 向晨

学号： 20221889

指导教师姓名： 韩强

企业指导教师： （无）

论文提交时间： 二〇二六年五月十五日

北方民族大学教务处制

北方民族大学本科毕业论文（设计）诚信承诺书

学生姓名	向晨	年级	2022 级
所学专业	软件工程	学号	20221889
所在学院	计算机科学与工程学院		

学生承诺

本人郑重承诺和声明：

我承诺在毕业论文（设计）过程中严格遵守学校有关规定，在指导教师的安排与指导下独立完成所规定的毕业论文（设计）工作，决不弄虚作假，不请别人代做毕业论文（设计）或抄袭别人的成果。所撰写的毕业论文或毕业设计是在指导老师的指导下自主完成，文中所有引文或引用数据、图表均注解并说明来源，本人愿意为由此引起的后果承担责任。

学生（签名）：_____

2026 年 05 月 日

基于 Vulkan 的实时光线追踪混合渲染器实现

摘要

在现代实时计算机图形学中，传统的纯光栅化渲染在处理全局光照、精确物理软阴影与复杂镜面反射时存在固有的物理局限性，而离线级别的纯路径追踪又难以满足交互式应用的实时帧率需求。针对这一“画质与算力”的矛盾，本文基于现代显式图形 API Vulkan 1.3，设计并实现了一套工业级的高性能实时光线追踪混合渲染引擎。

本文的核心工作包括三个方面。首先，针对 Vulkan 繁琐的显存状态同步痛点，设计了数据驱动的 Render Graph 调度架构，通过有向无环图的拓扑分析实现了管线屏障的自动推导与显存复用优化。其次，构建了高效的混合光照管线，将几何光栅化与硬件光线追踪深度解耦。在延迟渲染 G-Buffer 的基础上，结合偏导数面法线偏移与 Ray Query 扩展实现了精准无伪影的软硬阴影，并利用 RT Pipeline 全面求解 Cook-Torrance 微面元模型，完成了基于物理（PBR）的多次镜面反射计算。最后，针对 1 SPP 极低采样率带来的蒙特卡洛高频噪声，完整集成了时空方差导向滤波（SVGF）降噪系统。通过几何运动矢量的时域历史重投影与动态方差引导的多轮 A-Trous 空域滤波，成功实现了高保真度的抗噪平滑处理。

性能测试与效果分析表明，本系统在 2K 原生分辨率下，能够以实时的性能开销（稳定大于 60 帧）输出平滑且物理正确的照片级渲染画面。本文的研究验证了以 Render Graph 为底座、结合光栅化、硬件光追与 SVGF 降噪的混合架构在现代底层图形引擎开发中的高效性与工程可行性。

关键词: Vulkan; 实时光线追踪; 混合渲染; Render Graph; 基于物理的渲染 (PBR); SVGF 降噪

Implementation of a Real-time Ray Tracing Hybrid Renderer Based on Vulkan

ABSTRACT

In modern real-time computer graphics, traditional pure rasterization rendering has inherent physical limitations when dealing with global illumination, precise physical soft shadows, and complex specular reflections, while offline-level pure path tracing is difficult to meet the real-time frame rate requirements of interactive applications. To address this contradiction between "image quality and computing power", this paper designs and implements an industrial-grade high-performance real-time ray tracing hybrid rendering engine based on the modern explicit graphics API Vulkan 1.3.

The core work of this paper includes three aspects. First, a data-driven Render Graph scheduling architecture was designed to address the tedious GPU memory state synchronization pain points of Vulkan. Through topological analysis of a directed acyclic graph, the automatic derivation of pipeline barriers and optimization of memory reuse were achieved. Second, an efficient hybrid lighting pipeline was constructed to deeply decouple geometric rasterization from hardware ray tracing. Based on deferred rendering G-Buffer, combined with derivative face normal offset and Ray Query extension, precise and artifact-free soft and hard shadows were implemented. Furthermore, the RT Pipeline was used to comprehensively solve the Cook-Torrance microfacet model, completing multi-bounce specular reflection calculations based on physics (PBR). Finally, addressing the high-frequency Monte Carlo noise caused by the extremely low sampling rate of 1 SPP, a complete Spatiotemporal Variance-Guided Filtering (SVGF) denoising system was integrated. High-fidelity anti-noise smoothing was successfully achieved through temporal history reprojection of geometric motion vectors and multi-round spatial A-Trous filtering guided by dynamic variance.

Performance tests and effectiveness analysis show that this system can output smooth and physically correct photo-realistic rendering results with real-time performance overhead (stable above 60 FPS) at 2K native resolution. This research verifies the efficiency and engineering

feasibility of a hybrid architecture based on Render Graph, combining rasterization, hardware ray tracing, and SVGF denoising in modern low-level graphics engine development.

Key words: Vulkan; Real-time Ray Tracing; Hybrid Rendering; Render Graph; Physically Based Rendering (PBR); SVGF Denoising

目 录

摘要	II
ABSTRACT	III
第一章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状与发展	2
1.2.1 实时渲染框架的演进	2
1.2.2 光线追踪技术的发展	2
1.2.3 实时降噪与抗锯齿技术的发展	3
1.3 本文主要研究内容与贡献	3
1.4 论文结构安排	4
第二章 渲染管线与物理渲染理论	6
2.1 渲染管线的演进与对比	6
2.1.1 传统光栅化管线 (Rasterization)	6
2.1.2 实时混合渲染管线 (Hybrid Rendering)	6
2.2 基于物理的渲染 (PBR) 理论	6
2.2.1 渲染方程与能量守恒	6
2.2.2 微面元模型与 Cook-Torrance BRDF	7
2.3 现代图形 API 架构特性	7
2.3.1 显式同步与资源管理	7
2.3.2 动态渲染与管线状态对象 (PSO)	7
2.4 本章小结	7
第三章 渲染器架构与基础框架设计	8
3.1 渲染器概述与设计哲学	8
3.1.1 渲染的本质	8
3.1.2 渲染器与游戏引擎的关系	8
3.2 系统架构设计	8
3.2.1 架构参考与静态库模式	8

3.2.2	五层分层拓扑架构	9
3.3	模块化分层系统设计	10
3.3.1	Layer 的抽象定义	10
3.3.2	LayerStack: 层级管理与堆栈结构	10
3.3.3	执行序与事件拦截: 双向流动机制	10
3.3.4	事件驱动与输入轮询机制	11
3.3.4.1	事件系统 (Event System)	11
3.3.4.2	输入轮询 (Input Polling)	11
3.4	程序入口与主循环	12
3.4.1	入口点设计	12
3.4.2	游戏循环 (Game Loop)	12
3.5	资源抽象与句柄管理系统	12
3.5.1	资源句柄化与虚拟化 (Resource Handle)	12
3.5.2	基于 VMA 的显存池化管理	13
3.5.3	基于 Bindless 思想的全局描述符绑定	13
3.5.4	延迟销毁队列 (Deletion Queue)	13
3.6	场景组织与硬件加速结构维护	13
3.6.1	实体-组件式的场景结构	14
3.6.2	基于 GLTF 的场景导入流水线	14
3.6.3	硬件加速的层次包围盒 (BVH) 架构	14
3.7	交互式摄像机系统实现	14
3.7.1	摄像机原理: 操作倒转与观察空间	14
3.7.2	透视投影与裁剪空间	15
3.7.3	MVP 变换流水线	15
3.7.4	编辑器相机 (EditorCamera) 的交互实现	15
3.8	技术栈与构建方案	16
3.8.1	构建系统: CMake	16
3.8.2	核心依赖库清单	16
3.9	本章小结	16
第四章	渲染图架构与三大管线实现	17
4.1	渲染图调度系统设计	17
4.1.1	核心类层次结构与协作逻辑	17
4.1.2	声明式接口与自动屏障推导	19
4.1.3	基于闭包的跨管线调度	19
4.2	核心渲染管线的构建与实现	19

4.2.1	光栅化渲染管线 (Rasterization Pipeline)	19
4.2.2	光线追踪渲染管线 (Ray Tracing Pipeline)	20
4.2.3	混合渲染管线 (Hybrid Pipeline)	20
4.3	本章小结	23
第五章	后处理	24
5.1	硬件加速光线追踪的底层优化	24
5.1.1	加速结构 (AS) 与着色器绑定表 (SBT)	24
5.2	SVGF 时空方差引导降噪	24
5.2.1	反照率解耦 (Albedo Demodulation)	24
5.2.2	时域重投影与力矩累积	24
5.2.3	边缘停止函数的空域滤波	24
5.3	时域抗锯齿 (TAA) 与动态稳定性	25
5.3.1	亚像素抖动补偿与色彩裁剪 (Color Clipping)	25
5.4	后期影像流水线与电影级合成	26
5.4.1	Bloom 辉光与 HDR 合成	26
5.4.2	ACES 色调映射与色彩校正	26
5.5	本章小结	26
第六章	系统测试与分析	27
6.1	实验环境与方案设计	27
6.2	混合渲染质量评估	27
6.2.1	G-Buffer 中间数据准确性验证	27
6.2.2	光线追踪特效分析	28
6.2.3	混合渲染与传统光栅化对比	28
6.3	SVGF 降噪与 TAA 抗锯齿性能评估	28
6.3.1	SVGF 降噪稳定性分析	28
6.3.2	TAA 抗锯齿有效性验证	29
6.4	显存利用率优化与调试	29
6.4.1	基于 Render Graph 的显存别名 (Memory Aliasing)	29
6.4.2	显存安全与延迟销毁调试	29
6.5	系统集成界面与交互功能展示	29
6.6	本章小结	30
第七章	总结与展望	31
7.1	全文工作总结	31
7.2	未来研究方向	32

第一章 绪论

1.1 研究背景及意义

近三十年来，实时计算机图形学领域长期由光栅化（Rasterization）技术主导。光栅化凭借其卓越的并行计算效率，通过将三维几何顶点投影至二维屏幕空间并进行片元插值，能够在有限的硬件算力下实现高分辨率与高帧率的图像渲染。然而，光栅化本质上属于基于局部几何信息的启发式算法，难以精确模拟物理世界中光线在三维空间内的全局传输与多次反弹路径。因此，在处理全局光照（Global Illumination, GI）、基于物理的软阴影（Soft Shadows）以及多重粗糙镜面反射（Glossy Reflections）等复杂光学现象时，传统光栅化管线通常只能依赖预计算或屏幕空间经验近似算法。例如，使用屏幕空间环境光遮蔽（SSAO）来模拟暗角，使用级联阴影贴图（CSM）来计算硬阴影，以及使用屏幕空间反射（SSR）来模拟镜面效果。这些近似方案在复杂场景中极易引入明显的视觉伪影（Visual Artifacts），如 SSR 导致的屏幕外信息缺失与反射断裂。此外，随着现代数字娱乐产业对渲染真实感需求的日益提升，堆叠各类屏幕空间特效往往会导致渲染管线复杂度剧增，系统维护成本急剧上升。

随着图形硬件算力的持续突破以及 Vulkan Ray Tracing、DirectX Raytracing (DXR) 等现代图形 API 扩展的标准化，硬件加速的实时光线追踪（Hardware-Accelerated Ray Tracing）技术已正式步入实时渲染的舞台。该技术通过在硬件层面加速边界体积层次结构（Bounding Volume Hierarchy, BVH）的遍历与射线-三角形求交测试，能够精确求解渲染方程（Rendering Equation），从而生成具有照片级真实感（Photorealism）的图像。然而，受限于当前图形处理器（GPU）的显存带宽与核心算力瓶颈，在复杂的高分辨率实时交互场景中，若直接采用与离线影视渲染相同的纯路径追踪（Path Tracing）算法，每像素需要发射数千条光线，这与实时渲染严格的 16.6 毫秒（60 FPS）帧时间预算存在巨大的性能鸿沟。

面对这一性能与画质的尖锐矛盾，混合渲染（Hybrid Rendering）架构应运而生。混合渲染的核心思想是“扬长避短”：利用光栅化极高的可见性判定效率生成包含场景几何与材质属性的几何缓冲（G-Buffer），随后利用硬件光线追踪在 G-Buffer 的表面属性基础上发射极少量（通常为 1 SPP，即每像素 1 条射线）光线，专门用于求解光栅化难

以处理的阴影、反射和间接光照信号。然而，1 SPP 的极低采样率必然会导致渲染结果呈现出剧烈的蒙特卡洛随机噪声。因此，必须通过先进的时空降噪算法（Spatiotemporal Denoising）与时间抗锯齿技术（Temporal Anti-Aliasing, TAA）来修复低采样率带来的信噪比缺失。本文正是在此背景下，基于 Vulkan API 深入探索并实现了一套具备工业级降噪与抗锯齿能力的实时光线追踪混合渲染系统。

1.2 国内外研究现状与发展

实时渲染技术的发展史，本质上是对物理世界光照规律不断逼近的演进史。当前国内外的研究现状主要集中在以下三个技术脉络：

1.2.1 实时渲染框架的演进

早期的实时渲染引擎主要采用前向渲染（Forward Rendering）管线。前向渲染在处理每一个几何图元时，都会立即遍历场景中的所有光源进行光照计算。这种几何绘制与光照计算高度耦合的模式，导致其在面对多光源、高复杂度场景时，由于大量不可见片元被重复计算（Overdraw），性能会急剧下降。

为了解决多光源的性能瓶颈，延迟渲染（Deferred Rendering）被提出并广泛应用于现代 3D 引擎（如 Unreal Engine、Unity）中。延迟渲染通过引入几何缓冲（G-Buffer），将几何处理阶段与光照计算阶段完全分离。在几何阶段，系统仅将可见图元的反照率（Albedo）、法线（Normal）、材质粗糙度（Roughness）和深度（Depth）等属性写入 G-Buffer；在光照阶段，则直接利用 G-Buffer 中的物理属性进行全屏的光照合成。这种架构将光照计算的复杂度从“光源数量 $\times$ 几何图元数量”降低为“光源数量 $\times$ 屏幕像素数量”，极大提升了复杂光照环境下的渲染效率。然而，延迟渲染也带来了显存带宽开销大、难以处理半透明物体以及与传统多重采样抗锯齿（MSAA）不兼容等局限性。

1.2.2 光线追踪技术的发展

光线追踪技术最早由 Turner Whitted 于 1980 年提出（Whitted-style Ray Tracing），奠定了利用递归射线追踪实现镜面反射与折射的理论基础。1986 年，Kajiya 提出了著名的渲染方程（Rendering Equation），并将蒙特卡洛积分引入光线追踪，形成了路径追踪（Path Tracing）算法，首次在数学上统一了全局光照的求解模型。

长期以来，受限于庞大的计算量，该技术仅应用于影视特效等离线渲染领域。直到 2018 年，NVIDIA 发布了包含 RT Core 专用硬件加速单元的 RTX 系列显卡，Khronos Group 随后推出了 VK_KHR_ray_tracing 扩展集，实时光线追踪才在底层硬件和 API 层面成为可能。目前，国内外顶尖工业界的主流做法均是结合光栅化与光追的混合架

构，但如何在极低的光线预算下最大化光追画质，仍是学术界和工业界共同探索的热点方向。

1.2.3 实时降噪与抗锯齿技术的发展

在混合渲染管线中，降噪（Denoising）与抗锯齿（Anti-Aliasing）是决定最终画质的核心环节。

- **抗锯齿技术：**由于延迟渲染无法高效使用硬件 MSAA，早期工业界多采用快速近似抗锯齿（FXAA）或次像素形态抗锯齿（SMAA）等纯空间后处理算法，但这些算法无法解决高频几何边缘的时间闪烁问题。近年来，时间抗锯齿（Temporal Anti-Aliasing, TAA）逐渐成为主流。TAA 通过在投影矩阵中引入 Halton 亚像素抖动（Sub-pixel Jitter），结合运动矢量（Motion Vector）将多帧的历史采样结果进行指数滑动平均（EMA），以极低的性能开销实现了媲美超采样（SSAA）的抗锯齿质量。
- **降噪技术：**针对 1 SPP 极低样本率下的蒙特卡洛噪声，传统的空间降噪算法（如双边滤波或 Edge-avoiding $\hat{A}$ -Trous 小波滤波）在强力抹平噪声的同时，往往会严重模糊几何边界与纹理细节。2017 年，Schied 等人提出了时空方差导向滤波（SVGF, Spatiotemporal Variance-Guided Filtering）算法。该算法开创性地将时间域的历史累积与空间域的 $\hat{A}$ -Trous 滤波相结合，通过估算局部时间方差（Temporal Variance）来动态调整空间滤波的边缘停止权重（Edge-stopping weights）。SVGF 成功实现了在保留高频几何细节的同时抹平低频噪声，目前已成为业界处理实时光追信号的标准基线算法。

1.3 本文主要研究内容与贡献

本文针对现代实时渲染中高画质与高性能难以兼顾的问题，基于 Vulkan 图形 API 设计并实现了一套完整的实时光线追踪混合渲染系统。针对工业界开发中的痛点，本文在系统架构、混合管线设计以及时空信号处理上进行了深度优化。主要工作与贡献如下：

1. **设计并实现了基于 Render Graph 的渲染图架构：**针对 Vulkan 显式 API 中显存屏障（Memory Barrier）、图像布局转换（Image Layout Transition）和生命周期管理极其繁琐且易出错的痛点，实现了一套自动化的渲染图调度系统。该系统能够根据资源的读写约束和有向无环图（DAG）拓扑排序，在运行时自动推导管线的同步状态，极大提升了混合渲染管线的扩展性与资源复用率。
2. **构建了高效的混合物理光照渲染管线：**结合延迟渲染框架，在 G-Buffer 的基础上

实现了基于物理的渲染（PBR）光照计算。针对不同的光照特征灵活采用了底层光追技术：利用 Ray Query 结合几何偏导数（Derivative Face Normal）实现无自相交伪影的精确软硬阴影；利用 RT Pipeline（Raygen, Miss, ClosestHit）实现硬件加速的粗糙镜面反射与漫反射全局光照（GI）。

3. **实现了基于 SVGF 的工业级时空降噪系统：**针对 1 SPP 光追信号的严重噪声问题，实现并优化了 SVGF 降噪数据流。本文深入解决了 SVGF 算法在实际工程中遇到的多项隐蔽缺陷：包括修正 Reversed-Z（反转 Z 缓冲）非线性深度导致的边缘判定失效问题，以及提出在光追着色器中解耦反照率（Demodulate Albedo），从而避免空间滤波对纹理细节的错误涂抹。
4. **集成了高精度的 TAA 时间抗锯齿技术：**基于 Halton 序列实现了亚像素抖动系统。为解决 TAA 重投影过程中的历史错位与屏幕物理震动问题，本文在着色器底层推导并实现了严谨的抖动补偿公式（Jitter Compensation），并通过 AABT 颜色方差裁剪（Variance Clipping）有效消除了动态场景下的鬼影（Ghosting）现象，最终输出了极其平滑、稳定的照片级渲染画面。

1.4 论文结构安排

本文的组织结构如下：

- **第一章绪论：**阐述课题的研究背景、意义及国内外发展现状，并总结本文的主要研究内容与贡献。
- **第二章实时混合渲染理论基础：**详细介绍基于物理的渲染（PBR）材质模型、Vulkan API 的核心机制与显式同步理论，以及硬件光线追踪加速结构（BVH）的基本原理。
- **第三章基于 Render Graph 的渲染器架构设计：**详细论述渲染图的节点抽象、虚拟资源池管理、以及 Vulkan 状态屏障的自动推导算法与乒乓轮换（Ping-Pong）机制的工程实现。
- **第四章混合光照管线的核心设计与实现：**重点分析 G-Buffer 延迟管线构建、微表面 BRDF 物理光照合成逻辑，以及 Ray Query 与 RT Pipeline 在具体光影效果中的应用与优化。
- **第五章基于时空域的抗锯齿与降噪算法实现：**深入剖析蒙特卡洛噪声与锯齿的形成原理。详细推导 TAA 的亚像素重投影数学模型，并全面讲解 SVGF 算法中历史累积、方差估算与 λ -Trous 空域滤波的实现细节及工业级防坑指南。

- **第六章渲染效果展示与性能分析：**通过丰富的对比实验图展示混合渲染器的最终画面质量，对比光栅化与光线追踪的视觉差异，并通过 GPU 耗时统计验证系统在不同负载下的实时性能。
- **第七章总结与展望：**对全文的核心工作进行总结，客观分析当前系统的局限性，并对引擎未来可能的优化方向（如 ReSTIR 等新兴算法）进行展望。

第二章 渲染管线与物理渲染理论

本章主要阐述本系统实现的理论基础，包括渲染管线的演进逻辑、基于物理的渲染（PBR）数学模型以及现代图形 API 的架构特性。这些理论不仅定义了本系统“算什么”，更决定了在现代 GPU 架构下“怎么算”。

2.1 渲染管线的演进与对比

随着图形硬件算力的提升，实时渲染管线经历了从固定流水线到可编程流水线，再到如今光线追踪混合管线的重大变革。

2.1.1 传统光栅化管线 (Rasterization)

光栅化是将三维几何体投影到二维屏幕并离散化为像素的过程。其核心优势在于极高的并行效率，但在处理全局光照效果（如软阴影、多次反射、环境遮蔽）时，由于缺乏场景的全局拓扑信息，往往需要依赖各种经验性的近似算法（如 Shadow Mapping, SSR），这些算法在复杂场景下极易产生视觉伪影。

2.1.2 实时混合渲染管线 (Hybrid Rendering)

本系统采用混合渲染路径（Hybrid Rendering Pipeline）。其基本思路是：利用光栅化管线的高效性处理场景的几何可见性（写入 G-Buffer），而将昂贵的光线发射过程仅用于处理最能提升画质的光影特效。这种“先栅后追”的策略，是在当前硬件条件下实现次世代画质与实时帧率平衡的最优方案。

2.2 基于物理的渲染 (PBR) 理论

为了实现物理正确的材质表现，本系统严格遵循能量守恒定律与微面元模型。

2.2.1 渲染方程与能量守恒

所有光照计算的核心均围绕由 Kajiya 提出的渲染方程（The Rendering Equation）展开：

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (n \cdot \omega_i) d\omega_i \quad (2.1)$$

其中， f_r 为双向反射分布函数（BRDF），它定义了物体表面如何将入射光能重新分配到出射方向。能量守恒定律要求物体的反射总能量绝不能超过入射的总能量。

2.2.2 微面元模型与 Cook-Torrance BRDF

本系统将不透明物体表面建模为无数微小的镜面（Microfacets）。Cook-Torrance BRDF 将反射分为漫反射（Lambertian）与镜面反射（Specular）两部分。镜面反射项由以下三个关键物理因子组成：

- **法线分布函数 (D)**：描述微面元法线与半程向量对齐的概率，决定了高光斑的形状与锐利度。本系统采用 Trowbridge-Reitz GGX 分布。
- **菲涅尔项 (F)**：描述光线以不同角度入射时的反射比例，模拟掠射角下的全反射现象。采用 Schlick 近似模型。
- **几何遮蔽项 (G)**：描述微面元之间的相互遮掩（Masking）与阴影（Shadowing）效应，确保掠射角下的能量正确性。

2.3 现代图形 API 架构特性

本系统选择基于 Vulkan 1.3 开发，旨在利用其底层、显式且低开销的架构特性。

2.3.1 显式同步与资源管理

不同于旧式 API 的黑盒管理，Vulkan 要求开发者显式声明图像布局转换（Image Layout Transition）与执行屏障（Pipeline Barrier）。这种显式控制使得本系统实现的渲染图（Render Graph）模块能够精准地调度 GPU 任务，消除不必要的管线停顿（Stalls）。

2.3.2 动态渲染与管线状态对象 (PSO)

为了最大化渲染效率，本系统将深度测试、混合模式等所有渲染状态预编译为管线状态对象（PSO）。此外，利用 Vulkan 1.3 的动态渲染（Dynamic Rendering）特性，本系统彻底摆脱了复杂的 Render Pass 与 Framebuffer 对象绑定，极大地提升了渲染逻辑的灵活性。

2.4 本章小结

本章从渲染逻辑的演进出发，确立了混合渲染管线的技术路线；随后深入探讨了基于物理的渲染方程及其微面元实现模型，为后续着色器开发提供了数学指导；最后阐述了 Vulkan API 的核心特性，为底层的资源调度奠定了理论基础。关于如何利用硬件加速光线追踪的具体技术，将在第五章结合具体实现进一步展开。

第三章 渲染器架构与基础框架设计

本章将详细介绍 Chimera 渲染引擎的底层基础设施构建过程。首先从宏观视角阐述渲染器的基本定义及其在图形学系统中的位置，随后引出本项目参考的分层拓扑架构设计。本章还将深入探讨模块化分层系统、场景管理、资源调度以及交互式摄像机系统的实现细节，并总结项目所采用的技术栈。

3.1 渲染器概述与设计哲学

3.1.1 渲染的本质

渲染（Rendering）是三维计算机图形学中利用软件从三维模型生成二维图像的过程。在这一过程中，计算机需要对三维模型或场景进行一系列复杂的处理，包括建模、纹理映射、光照计算、投影变换以及视点变换等，最终在光栅化阶段生成像素阵列。

渲染器（Renderer）是游戏引擎中最核心的子系统之一。它负责将抽象的游戏世界数据（如模型顶点、纹理采样器、光照参数等）转换为最终呈现在屏幕上的图像。从架构角度看，渲染器可以被理解作为一种“翻译官”或“中间层”——它将游戏世界的逻辑状态翻译为 GPU 可执行的底层图形操作。

3.1.2 渲染器与游戏引擎的关系

相比于一个完整的游戏引擎，渲染器只是庞大系统架构中的一个子集。在本项目中，我剥离了与图形学研究无关的业务逻辑（Gameplay）、物理引擎和音频系统，专注于打造一个纯粹的渲染实验平台。

然而，在底层架构上，高质量的渲染器与现代游戏引擎有着极高的相似度。即便不考虑游戏逻辑，一个完备的渲染器依然需要具备模型加载、资源管理、事件响应等基础功能。因此，在搭建 Chimera 时，我也参考了通用游戏引擎的工业级架构方案。

3.2 系统架构设计

3.2.1 架构参考与静态库模式

本项目的架构设计深受 Jason Gregory 所著《游戏引擎架构》（Game Engine Architecture）的影响，并参考了 TheCherno 开发的 Walnut 引擎。

在工程实现上，Chimera 采用了“引擎核心 + 沙盒应用”的分离模式：

- **引擎核心 (Chimera)**: 被编译为一个静态库（Static Library）。它封装了所有复杂

的图形 API 调用和渲染调度逻辑。

- **沙盒应用 (Sandbox):** 作为一个独立的的可执行项目，通过调用引擎库提供的 API 来编写具体的渲染指令。

这种设计为用户提供了一个不受干扰的开发环境。在 Sandbox 中，开发者可以像在成熟引擎中写脚本一样编写渲染代码，而无需关心底层的交换链管理或同步屏障。

3.2.2 五层分层拓扑架构

为了提高系统的可扩展性并降低模块间的耦合度，Chimera 采用了清晰的分层拓扑架构。整个引擎从顶层应用到底层硬件共划分为五个核心逻辑层：

1. **应用层 (Sandbox Layer):** 具体的沙盒应用入口。通过继承 Application 基类并挂载自定义的 Layer，开发者可以快速构建不同的渲染场景，实现业务逻辑与引擎核心的解耦。
2. **核心框架层 (Core Layer):**
 - **LayerStack:** 支持“插拔式”的模块管理，如处理 3D 渲染的 SceneLayer 和处理 UI 的 ImGuiLayer。
 - **事件系统:** 采用“从顶到底”的事件拦截机制，确保 UI 操作不会误触发场景交互。
 - **日志系统:** 集成 spdlog，实现带颜色分级的异步日志记录，辅助实时监控引擎状态。
3. **场景与资源层 (Scene & Assets Layer):**
 - **加速结构 (AS):** 负责实时更新光线追踪所需的 TLAS（顶层加速结构）和 BLAS（底层加速结构）。
 - **ResourceManager:** 通过句柄 (Handle) 管理显存资源，并利用延迟销毁队列确保在 GPU 繁忙时不会误删资源。
4. **渲染调度层 (Renderer Layer):**
 - **RenderGraph:** 核心调度中枢，通过 DAG（有向无环图）自动推断 Pass 间的依赖关系。
 - **资源抽象:** 封装了纹理 (Image)、缓冲区 (Buffer) 及着色器 (Shader) 等低级对象的生命周期。
5. **平台与硬件层 (Platform Layer):** 直接与操作系统交互。
 - **Vulkan 环境:** 利用 volk 动态加载 API 入口，负责逻辑设备、交换链及验证

层的配置。

- **窗口管理：**通过 GLFW 捕获原生输入事件并转发至核心层。

3.3 模块化分层系统设计

在复杂的实时渲染引擎开发中，降低系统耦合度与控制逻辑复杂度是首要任务。为此，Chimera 引入了源自 Hazel 引擎设计的“分层系统”（Layer System）。这一机制将渲染、逻辑更新与事件响应解构为多个独立的原子单元，极大地提升了代码的组织效率。

3.3.1 Layer 的抽象定义

Layer（层）是一个高度抽象的逻辑概念，它代表了引擎中负责特定呈现内容或逻辑的模块。在 Chimera 中，Layer 基类定义了四个核心生命周期钩子：

- **OnAttach / OnDetach：**当层被推入或移出栈时触发，用于资源的初始化与释放。
- **OnUpdate：**每帧执行一次，用于处理该层特有的逻辑更新（如相机移动、场景动画）。
- **OnImGuiRender：**专门用于提交 ImGui 绘图指令，将调试 UI 与核心场景渲染逻辑分离。
- **OnEvent：**接收来自应用层的分发事件，并根据逻辑决定是否拦截。

在实际工程中，我通常将系统划分为 SceneLayer（处理 3D 场景数据）和 ImGuiLayer（处理编辑器面板与性能监控）。这种划分使得我在修改 UI 布局时，完全不会干扰到核心渲染管线的执行。

3.3.2 LayerStack：层级管理与堆栈结构

为了高效管理这些层，引擎实现了 LayerStack 类。它在内部维护了一个连续的容器，并引入了“普通层（Layer）”与“覆盖层（Overlay）”的区分：

- **普通层（Layer）：**按照添加顺序存储在容器的前半部分。例如，底层的环境背景层先添加，上层的模型层后添加。
- **覆盖层（Overlay）：**始终被维护在容器的最末端。它们通常具有最高优先级，用于显示始终置顶的调试信息、性能 Overlay 或全局弹出通知。

3.3.3 执行序与事件拦截：双向流动机制

分层系统的核心精髓在于其对渲染流与事件流的差异化处理：

1. **渲染流（从底向上，Bottom-to-Top）：**在 OnUpdate 阶段，引擎从 LayerStack 的最底层（索引 0）开始遍历。这样做可以确保背景先绘制，模型次之，最后是覆盖在最上方的 UI 界面。这种正向遍历符合图形学的深度覆盖逻辑。

2. **事件流（从顶向下，Top-to-Bottom）**：当鼠标点击或按键触发时，事件分发器会从最顶层的 Overlay 开始倒序遍历（从栈顶到栈底）。

这种“从顶向下”的事件传递机制实现了关键的事件拦截（Event Interception）。例如，当用户点击 ImGui 面板上的一个按钮时，顶层的 ImGuiLayer 会捕获该点击事件并将其标记为“已处理（Handled）”。此时，下层的 SceneLayer 将不再收到该事件，从而有效避免了“穿透点击”现象——即在操作 UI 菜单时意外触发了场景中物体的选择或移动。

3.3.4 事件驱动与输入轮询机制

为了实现平滑且高性能的交互响应，Chimera 引擎设计了一套结合“事件驱动（Event-Driven）”与“输入轮询（Input Polling）”的混合交互架构。

3.3.4.1 事件系统（Event System）

事件系统主要用于处理“一次性”的动作，如窗口缩放、鼠标点击或按键按下。所有事件类均继承自 Event 基类，并按功能划分为 KeyEvent、MouseEvent 与 ApplicationEvent 等类别。

事件处理的核心中枢是 EventDispatcher（事件分发器）。它的本质是一个基于模板的路由机制：

- **按需拦截**：分发器通过 Dispatch<T> 方法尝试将通用事件匹配为特定类型（如 MouseScrolledEvent）。
- **消耗机制**：与之绑定的回调函数会返回一个布尔值。如果返回 true，则表示该事件已被当前 Layer “消耗（Consume）”，分发过程立即停止。

如前文所述，事件流遵循“从顶向下”的穿透逻辑。位于栈顶的 Overlay（如 UI 界面）最先获得处理事件的机会。这种设计确保了 UI 能够优先截获输入，防止点击 UI 按钮时意外触发场景中相机的旋转，从而完美解决了“事件穿透”导致的冲突问题。

3.3.4.2 输入轮询（Input Polling）

虽然事件系统能够精准捕获动作，但对于需要“连续平滑响应”的交互（如 WASD 相机漫游），纯事件驱动往往会受限于操作系统的按键重复延迟（Key Repeat Delay），产生明显的顿挫感。

为此，引擎引入了输入轮询机制。通过静态类 Input，系统可以直接查询当前硬件的瞬时状态：

- **主动查询**：在 OnUpdate 逻辑中，代码可以直接调用 Input::IsKeyPressed(Key::W)。

- **状态同步**：该接口本质上是对底层 GLFW 输入状态的封装，能够以与帧率同步的频率获取输入，从而实现无延迟的平滑移动体验。

综上所述，事件系统负责“决策与切换”（如切换渲染路径、开关 TAA），而输入轮询负责“连续交互与操作”。两者的结合使得 Chimera 能够同时具备响应的精确性与操作的丝滑感。

3.4 程序入口与主循环

3.4.1 入口点设计

Chimera 使用了游戏开发中经典的入口点（Entry Point）模式。开发者不需要编写 main 函数，而是通过宏定义将自定义的 Application 子类注入引擎。

3.4.2 游戏循环 (Game Loop)

在 Application::Run() 方法中，引擎维持着一个高效的死循环。每一帧的操作严格遵循以下序：

1. **获取步长**：计算两帧之间的时间差（Timestep），用于物理平滑交互。
2. **事件轮询**：通过操作系统获取最新的鼠标和键盘状态。
3. **层级更新**：遍历 LayerStack，执行每一层的 OnUpdate 逻辑。
4. **渲染提交**：调用 Renderer 提交当前帧的绘制命令缓冲。
5. **UI 渲染**：在场景渲染完成后，叠加 ImGui 调试界面。

3.5 资源抽象与句柄管理系统

在 Vulkan 这种显式图形 API 中，资源管理（显存分配、生命周期追踪与描述符绑定）是渲染器开发中最具挑战性的模块。为了确保系统的健壮性并降低 CPU 端的调度开销，Chimera 实现了一套高度抽象的“句柄化资源调度系统”。

3.5.1 资源句柄化与虚拟化 (Resource Handle)

为了规避 C++ 原始指针带来的悬空指针风险与显存泄露问题，引擎引入了句柄化访问模式。逻辑层不再直接持有 VkBuffer 或 VkImage，而是通过轻量级的 TextureHandle、MaterialHandle 等句柄进行索引。

这一设计的核心价值在于：

- **地址隔离**：ResourceManager 内部维护着连续的资源池。即便底层物理资源因动态扩容或内存整理而发生位移，上层逻辑层持有的句柄 ID 依然保持不变，实现了逻辑引用与物理地址的解耦。
- **类型安全**：通过强类型的句柄包装，系统能在编译期拦截错误的资源操作（如将材

质句柄误传给纹理采样函数)。

3.5.2 基于 VMA 的显存池化管理

在底层显存分配上,本项目集成了 AMD 开发的 Vulkan Memory Allocator (VMA)。由于 Vulkan 物理设备对内存分配次数有严格限制(通常仅为 4096 次),直接为每个纹理调用 `vkAllocateMemory` 会迅速耗尽系统资源。VMA 通过在大块预分配内存(Heap)上进行子分配(Sub-allocation)和碎片整理,实现了高效的显存池化管理。Chimera 将其封装在 Buffer 与 Image 类中,极大地简化了 Staging Buffer 的创建与数据同步流程。

3.5.3 基于 Bindless 思想的全局描述符绑定

为了支持百万级面片与海量纹理的实时混合渲染,Chimera 采用了现代化的 ****Bindless**** 资源绑定架构:

1. **材质存储缓冲区(SSBO):** `ResourceManager` 会将场景中所有材质的 PBR 参数(颜色、金属性、纹理索引等)打包成一个连续的 `GpuMaterial` 数组,并通过 `SyncMaterialsToGPU` 一次性上传至全局存储缓冲区。
2. **全局纹理数组:** 引擎维护了一个巨大的描述符数组(Descriptor Array),将场景中所有的纹理视图(Image View)一次性绑定至全局 Descriptor Set。

在着色器中,程序不再需要频繁切换纹理绑定点,而是通过材质结构体中存储的索引值,直接在全局数组中访问对应的纹理。这种设计彻底消除了光栅化时代“一个物体一个 Draw Call”的性能瓶颈。

3.5.4 延迟销毁队列 (Deletion Queue)

Vulkan 严禁在 GPU 仍在执行渲染指令时销毁其引用的资源。为了解决 CPU-GPU 异步导致的资源竞争问题,Chimera 实现了一套延迟销毁机制(Deletion Queue):当用户请求释放某个资源时, `ResourceManager` 并不会立即调用销毁函数,而是将其推入队列并标记当前的帧序号。系统会等待两到三帧(对应双重或三重缓冲的完整循环)后,确保 GPU 已经彻底释放了对该资源的引用,才会执行最终的物理清理。这一机制是引擎能够稳定运行、不发生显存溢出的关键保障。

3.6 场景组织与硬件加速结构维护

如果说资源系统提供了渲染的“砖块”,那么场景系统则定义了“建筑的拓扑结构”。在实时光线追踪背景下,传统的空间划分方案(如八叉树或 BSP 树)因其结构僵硬且 CPU 维护开销大,已逐渐被更适合 GPU 遍历的层次包围盒(BVH)所取代。

3.6.1 实体-组件式的场景结构

为了兼顾开发灵活性与运行效率，Chimera 的场景层采用了轻量级的组件化设计：

- **Entity（实体）**：场景中的基本逻辑单元，持有一个全局唯一的 ID。
- **TransformComponent**：负责维护物体的 4x4 TRS 变换矩阵，并追踪“前一帧变换（Previous Transform）”，为 TAA 和运动模糊提供关键的运动矢量数据。
- **MeshComponent**：作为几何体载体，持有关联的 Model 句柄，实现了几何数据与逻辑实体的解耦。

3.6.2 基于 GLTF 的场景导入流水线

场景的数据驱动主要通过 GLTF 2.0 格式实现。本项目利用 cgltf 库实现了对模型层级（Node Hierarchy）的深度解析。加载流程不仅提取顶点数据，还递归地计算节点间的父子变换关系，最终构建出一棵符合物理逻辑的场景树。

3.6.3 硬件加速的层次包围盒（BVH）架构

为了支撑百万级面片的实时光线追踪，Scene 类最核心的职责是维护一套符合 Vulkan KHR 规范的双层加速结构（Acceleration Structure）：

1. **底层加速结构（BLAS）**：对应单个模型的 BVH。在模型加载时，驱动程序根据原始三角形的 AABB 自动构建最优的二叉树结构。BLAS 存储在显存中，作为静态的几何描述符。
2. **顶层加速结构（TLAS）**：对应整个场景的 BVH。它本质上是一个轻量级的“实例引用表（Instance List）”。

这种“两层拓扑”的设计实现了极高的更新效率：当场景中的物体（Entity）发生平移或旋转时，其内部的几何细节（BLAS）无需变动，引擎只需在 `Scene::UpdateTLAS` 中更新对应的 4x4 变换矩阵并重构轻量级的 TLAS 节点。这种基于硬件加速的 BVH 遍历机制，使得射线与场景的求交测试（Ray-Box Intersection）能够完全由 GPU 的 RT Core 异步完成，大幅降低了 CPU 的调度负载。

3.7 交互式摄像机系统实现

在三维渲染中，摄像机（Camera）是用户观察虚拟世界的“眼睛”。然而，在底层图形学实现中，摄像机实体并不物理存在，它本质上是一系列坐标空间变换的数学抽象。

3.7.1 摄像机原理：操作倒转与观察空间

理解摄像机的核心在于“逆变换”逻辑。在虚拟世界中，我们习惯于认为相机在空间中移动，但在渲染流水线中，实际发生的是 ** 整个场景相对于相机的反向运动 **。

例如，当用户指令相机向左移动时，渲染引擎实际上是将场景中所有的物体向右平移。这种将物体从世界空间（World Space）转换到观察空间（View Space）的变换矩阵被称为 **** 观察矩阵（View Matrix）****。它代表了相机位置与姿态的逆矩阵，决定了我们如何“看待”这个场景。

3.7.2 透视投影与裁剪空间

为了将观察空间的三维数据映射到二维屏幕上，我们需要引入 **** 投影矩阵（Projection Matrix）****。本项目主要采用透视投影（Perspective Projection），其核心参数包括：

- **视场角（FOV）**：决定了视野的广度。较大的 FOV 会产生广角效果，增强透视感。
- **纵横比（Aspect Ratio）**：视口的宽度与高度之比，确保图像不会因窗口拉伸而变形。
- **近/远裁剪面（Near/Far Clip）**：定义了相机可观察的深度范围，超出此范围的几何体将被裁剪，以节省计算资源并防止深度冲突（Z-Fighting）。

3.7.3 MVP 变换流水线

一个顶点从模型原始数据到最终屏幕像素，需要经过完整的 MVP 变换链：

$$V_{clip} = P \cdot V \cdot M \cdot V_{local} \quad (3.1)$$

其中：

1. **模型矩阵（M）**：通过平移、旋转、缩放（TRS）将顶点从局部坐标系转换到世界坐标系。
2. **观察矩阵（V）**：将世界坐标系中的物体转换到以相机为原点的观察坐标系。
3. **投影矩阵（P）**：应用透视除法，将坐标转换到归一化设备坐标（NDC）空间。

3.7.4 编辑器相机（EditorCamera）的交互实现

Chimera 实现了一套基于弧球（Arcball）逻辑的编辑器相机。与传统的 FPS 相机不同，它围绕一个 **** 焦点（Focal Point）**** 进行观察。

- **轨道飞行（Orbit）**：通过计算鼠标移动产生的偏航角（Yaw）和俯仰角（Pitch），结合四元数旋转，实现围绕目标的平滑旋转。
- **自动缩放与平移**：根据相机到焦点的距离动态调整移动步长，确保在观察宏观场景或微观细节时都能保持一致的操作手感。

此外，为了适配 Vulkan 的坐标系规范，本系统在投影矩阵中修正了 Y 轴的方向，并采用了 Reversed-Z 技术（近平面映射为 1.0，远平面映射为 0.0），以利用浮点数在零

点附近的更高精度，从而在渲染大规模场景时极大地缓解了深度冲突问题。

3.8 技术栈与构建方案

3.8.1 构建系统：CMake

项目选择 **CMake** 作为构建系统。由于 Chimera 依赖大量的开源第三方库（如 GLFW, spdlog, GLM），CMake 的 `add_subdirectory` 配合 `git submodule` 能够极大地简化依赖管理，并保证跨平台的编译一致性。

3.8.2 核心依赖库清单

- **Vulkan SDK 1.3**: 提供现代化的硬件访问接口。
- **volk**: Vulkan 的元装载器，用于动态获取函数指针。
- **Dear ImGui**: 用于构建实时调试面板。
- **VMA**: 由 AMD 提供的显存分配器，解决 Vulkan 原生内存分配次数受限的问题。
- **cgltf**: 轻量级 GLTF 2.0 解析器。

3.9 本章小结

本章构建了 Chimera 引擎的架构骨架。通过五层分层架构实现了系统解耦，并确立了“引擎库 + 应用”的工程模式。这些基础设施——包括 Application 主循环、资源管理句柄以及交互式相机——为后续章节探讨复杂的混合渲染管线奠定了坚实的工程基础。

第四章 渲染图架构与三大管线实现

本章将深入探讨 Chimera 引擎的核心调度中枢——渲染图（Render Graph）模块，并详细阐述如何基于该架构构建光栅化、光线追踪以及混合渲染三大核心管线。通过声明式的资源管理与自动化的同步机制，本系统实现了复杂渲染流程的高效调度。

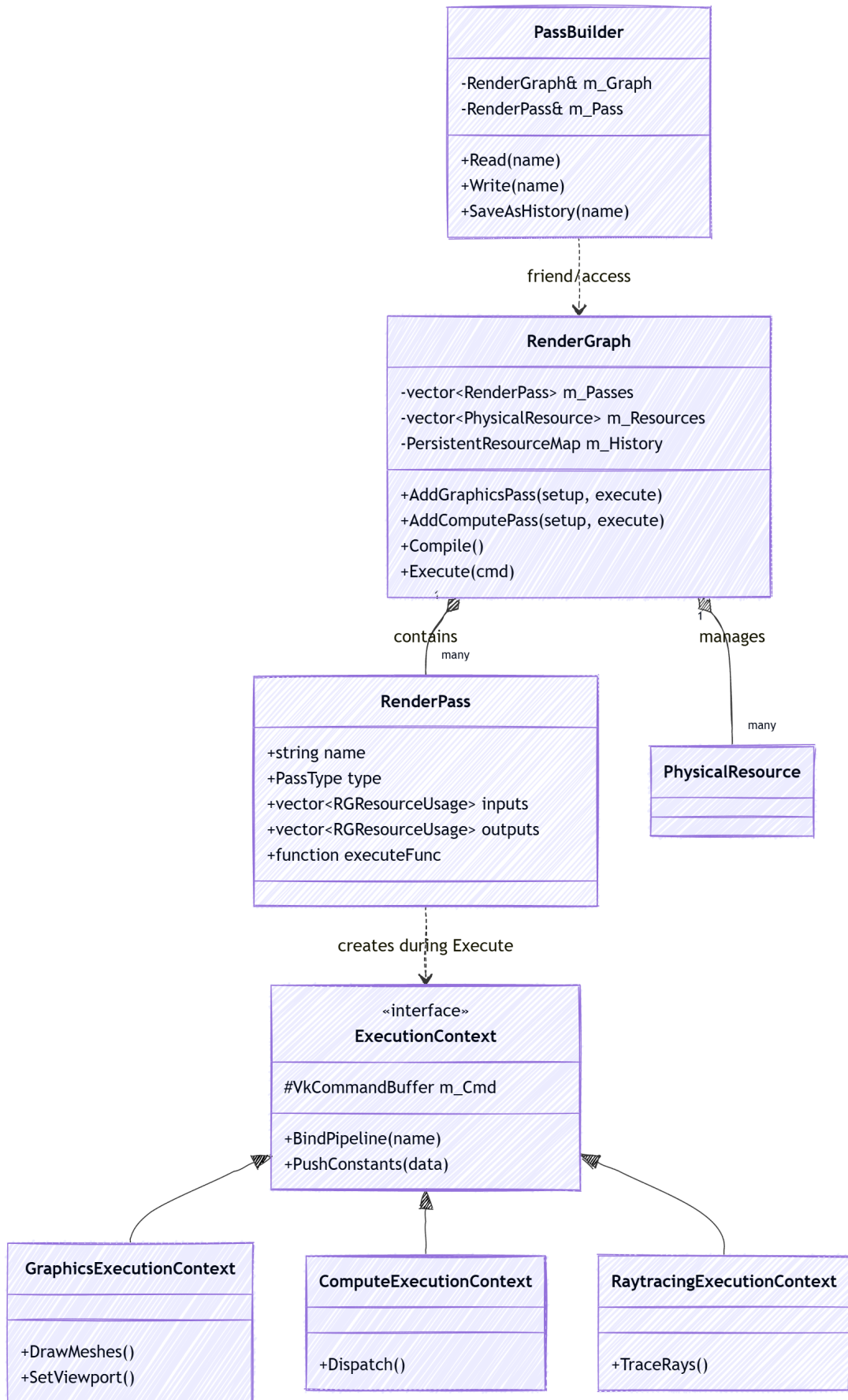
4.1 渲染图调度系统设计

Render Graph 是 Chimera 引擎的调度中枢，其核心目标是将高层的渲染逻辑与底层的 Vulkan 资源同步（如屏障、布局转换）解耦。通过分析其源码实现，本系统的 Render Graph 采用了一种“声明式”与“流式接口”相结合的设计模式。

4.1.1 核心类层次结构与协作逻辑

渲染图系统的实现涉及多个核心类，如图 4.1 所示。其协作逻辑可归纳为以下三个阶段：

1. **声明阶段 (Setup Phase)**: 利用 PassBuilder 提供流式 API。通过 AddPass 方法，开发者在 Lambda 闭包中定义资源的读写关系。
2. **代理配置 (Resource Proxy)**: ResourceHandleProxy 允许在声明时链式配置资源的格式（Format）、清空值（Clear Value）以及持久化属性（Persistent），这体现了 Builder 设计模式的灵活性。
3. **执行阶段 (Execution Phase)**: 当 Execute 被触发时，RenderGraphRegistry 作为虚拟句柄与物理 Vulkan 对象（如 VkImageView）之间的桥梁，为闭包提供真实的硬件资源。

18
图 4.1 Render Graph 核心类层次结构图

4.1.2 声明式接口与自动屏障推导

Render Graph 的核心价值在于将 Vulkan 繁琐的屏障（Barrier）管理自动化。开发者只需声明 Pass 对资源的读写意图（如 Read 或 Write），系统会自动对比资源的当前布局（Layout）并插入 vkCmdPipelineBarrier2。这解决了手动管理写后读（RAW）冒险时极易出错的问题，使开发者能专注于渲染算法本身。

4.1.3 基于闭包的跨管线调度

为了统一处理图形（Graphics）、计算（Compute）与光线追踪（Ray Tracing）任务，本系统设计了一套策略模式的执行上下文（ExecutionContext）。

- **解耦机制：**RenderPass 类不直接持有具体的绘制指令，而是持有一个执行闭包。
- **多管线支持：**根据 Pass 类型的不同，系统会向闭包注入不同类型的上下文，从而实现如模型绘制（DrawMeshes）、计算调度（Dispatch）或光迹追踪（TraceRays）的统一调用。

4.2 核心渲染管线的构建与实现

在 Render Graph 的框架下，本系统构建了三条具有不同应用场景的渲染路径。

4.2.1 光栅化渲染管线 (Rasterization Pipeline)

光栅化路径利用 Vulkan 动态渲染特性实现高效的前向渲染。在 Chimera 引擎中，该路径的资源流转拓扑如图 4.2 所示，体现了从几何绘制到后期处理的完整线性流水线。

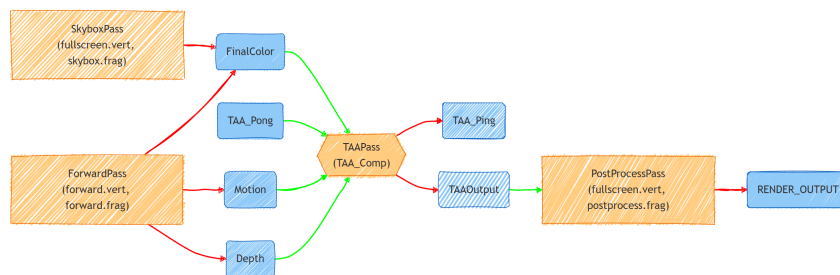


图 4.2 前向光栅化管线资源流转拓扑图

根据拓扑图分析，光栅化管线的执行逻辑可归纳为以下步骤：

1. **几何与环境绘制阶段：**SkyboxPass 与 ForwardPass 并行或顺序写入 FinalColor 附件。其中，ForwardPass 除了输出颜色外，还负责生成 Depth（深度缓冲）和 Motion（运动矢量），这些是后续时域算法的关键输入。
2. **时域抗锯齿 (TAA) 阶段：**利用计算管线实现的 TAAComp 接收原始颜色、深度及

运动矢量，并结合上一帧的历史资源（TAA_Pong）进行累积采样。该阶段输出平滑后的 TAAOutput 并更新历史记录（TAA_Ping）。

3. **后期处理与输出阶段:** PostProcessPass 对 TAA 处理后的信号执行色调映射（Tone Mapping）与伽马校正，最终生成 RENDER_OUTPUT 提交至交换链进行显示。

4.2.2 光线追踪渲染管线 (Ray Tracing Pipeline)

利用显卡的硬件加速单元，系统实现了完整的光路追踪路径。与光栅化管线不同，该路径的核心在于利用着色器绑定表（SBT）组织的递归追踪流程。其资源流转拓扑如图 4.3 所示。

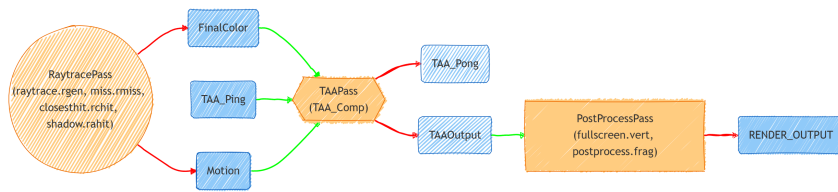


图 4.3 纯光线追踪管线资源流转拓扑图

光线追踪管线的逻辑流程如下：

1. **光线生成与求交阶段:** 核心节点 RaytracePass 整合了射线生成（.rgen）、未命中（.rmiss）、最近交点（.rchit）及阴影任意交点（.rahit）四类着色器。该阶段直接根据相机参数发射射线，并输出原始光照颜色 FinalColor 与用于时域对齐的 Motion 矢量。
2. **时域稳定阶段:** 由于实时光追通常限制在极低采样率（如 1 SPP），输出画面存在严重的噪声。系统通过 TAA Pass 引入时域反馈，利用历史缓冲区 TAA_Ping 与当前帧进行加权融合，生成相对稳定的 TAAOutput。
3. **后期处理阶段:** PostProcessPass 接收稳定后的信号，完成 HDR 到 SDR 的转换，最终输出 RENDER_OUTPUT。

4.2.3 混合渲染管线 (Hybrid Pipeline)

混合管线结合了光栅化与光线追踪的优势，是本系统的核心渲染模式。其设计理念是利用光栅化处理高带宽的几何可见性，利用光线追踪处理复杂的非局部光照特效。

根据图 4.4 的拓扑结构，混合管线在不开启降噪时的执行逻辑如下：

1. **G-Buffer 几何采集阶段:** GBufferPass 利用光栅化管线一次性输出 Albedo、Normal、Material、Motion、Emissive 及 Depth。这些高精度的几何与材质信号为后续的光追阶段提供了完美的“底座”。

2. 分路光线追踪阶段：系统并行发起多路光追任务。

- RTShadowAOPass：基于深度与法线生成硬/软阴影及环境遮蔽（Shadow/AO）。
- RTReflectionPass：利用材质参数追踪镜面反射信号，输出 `ReflectionRaw`。
- RTDiffuseGIPass：计算场景间的间接弥散光照，输出 `GIRaw`。

3. 物理合成与稳定阶段：Composition Pass 接收上述所有光栅化与光追结果，按照物理光照模型进行最终合并（`FinalColor`）。最后通过 `TAA Pass` 执行时域抗锯齿，并进入后期处理流程。

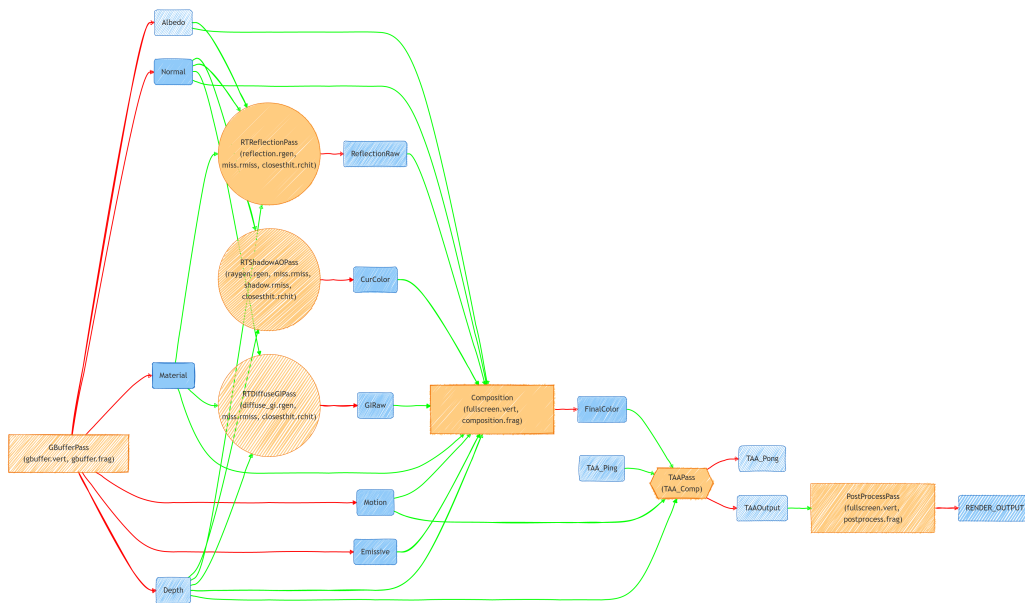


图 4.4 混合渲染管线资源流转拓扑图 (关闭 SVGF 降噪)

而在开启 SVGF 降噪后，如图 4.5 所示，管线的复杂度显著增加。系统为每一路光追信号（阴影、反射、全局光照）分别构建了独立的降噪流水线：

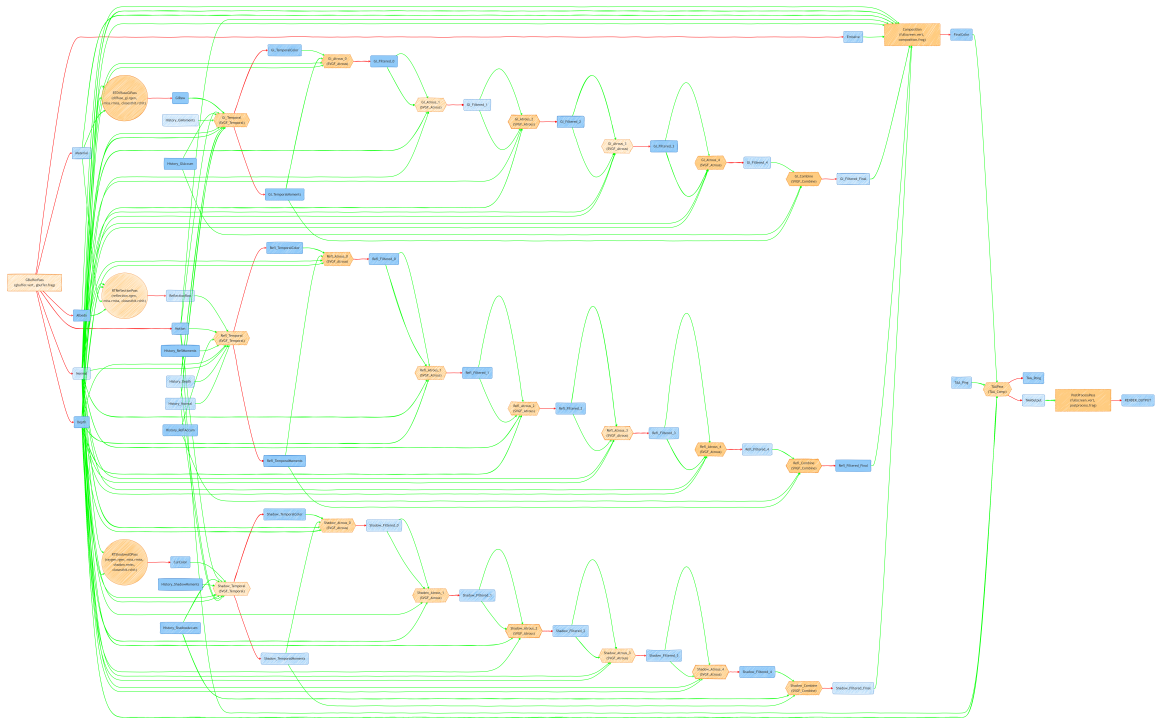
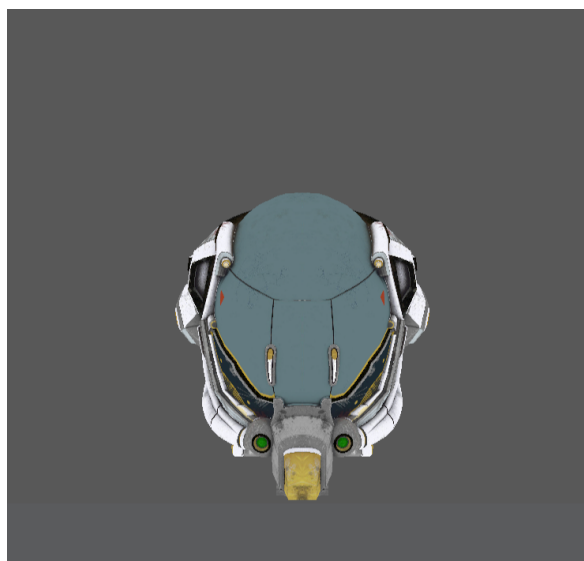


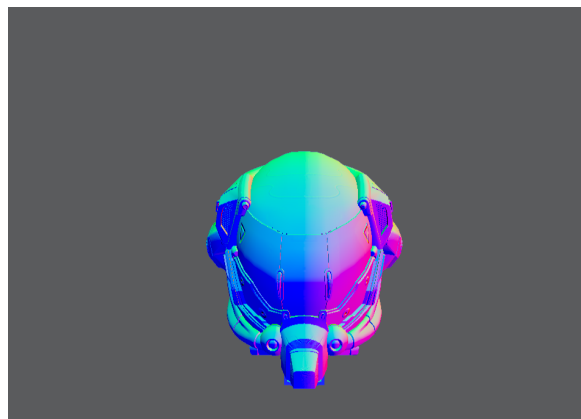
图 4.5 混合渲染管线资源流转拓扑图 (开启 SVGF 降噪)

- **多路并行降噪 (Multi-channel Denoising):** 针对 CurColor (阴影/AO)、ReflectionRaw 和 GIRaw, 系统并行启动了三组 SVGF 逻辑。每一组均包含时域累积 (Temporal)、五级阶梯式 A-Trous 滤波 (Atrous 0-4) 以及最终合并 (Combine) 三个子阶段。
- **时域反馈环路 (History Feedback Loop):** 这是拓扑图中最为关键的结构。在 Temporal 阶段, 系统不仅读取当前帧的 G-Buffer, 还深度依赖于上一帧保留的 History_Depth、History_Normal 以及各路信号的历史累积值 (如 History_ShadowAccum)。这种跨帧的数据流转形成了图中的闭环, 是实现 1 SPP 稳定渲染的核心。
- **边缘保留滤波 (Edge-stopping Filtering):** 在五次迭代的 A-Trous 滤波过程中, 系统利用 Normal 和 Depth 计算边缘权重, 确保光照信号在平滑的同时, 几何边缘 (如墙角、物体轮廓) 依然清晰, 避免产生“漏光”或“糊影”现象。

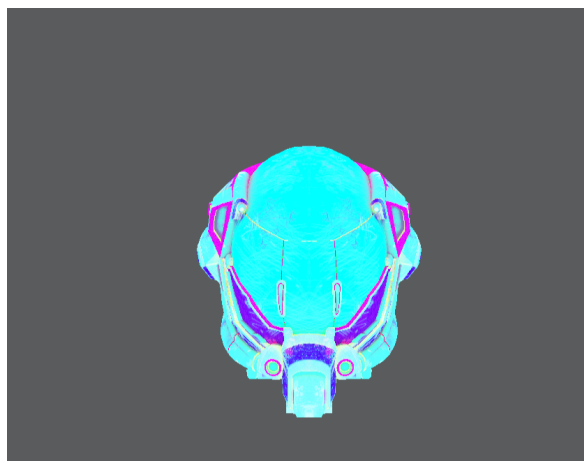
最终, 经过降噪处理后的高质量信号 (如 Shadow_Filtered_Final) 被送入 Composition Pass。这种设计确保了即便在光追采样极其稀疏的情况下, 依然能获得具备时域稳定性且物理正确的渲染效果。



(a) 反照率 (Albedo)



(b) 世界空间法线 (Normal)



(c) 材质属性 (Material)



(d) 线性深度 (Linear Depth)

图 4.6 混合渲染管线底座：G-Buffer 中间层输出结果

4.3 本章小结

本章完成了 Chimera 引擎核心调度架构与三大渲染管线的构建。Render Graph 的引入不仅解决了 Vulkan 复杂的同步难题，更为光栅化与光线追踪的深度融合提供了底座。这使得系统能够在下一章中，专注于解决 1 SPP 采样率下的信号重建与画质增强难题。

第五章 后处理

本章将深入探讨在实时光线追踪低采样率（1 SPP）条件下，如何通过时空降噪与后期处理算法实现画质重建。重点阐述 SVGF 降噪网络、TAA 抗锯齿以及电影级色调映射的具体实现与调优过程。

5.1 硬件加速光线追踪的底层优化

在混合管线的底座之上，本章利用硬件级光路追踪求解高精度的间接光效应。

5.1.1 加速结构 (AS) 与着色器绑定表 (SBT)

为了实现高效的射线求交，系统构建了两级加速结构：底层（BLAS）存储网格几何，顶层（TLAS）处理实例变换。通过构建着色器绑定表（SBT），系统实现了对 RayGen、Miss 和 Closest Hit 着色器的灵活调度，从而支持了物理正确的反射与全局光照计算。

5.2 SVGF 时空方差引导降噪

实时光线追踪生成的图像伴随严重的蒙特卡洛噪声。本系统实现了工业级的 SVGF 算法。

5.2.1 反照率解耦 (Albedo Demodulation)

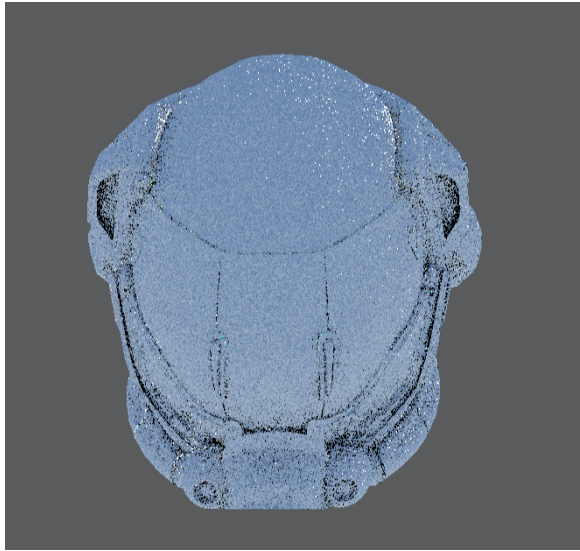
为了防止空域滤波模糊掉材质纹理，系统在滤波前将光照颜色除以反照率 (Albedo)。这一策略确保了滤波过程仅作用于纯粹的光照信号，保留了高频纹理细节。

5.2.2 时域重投影与力矩累积

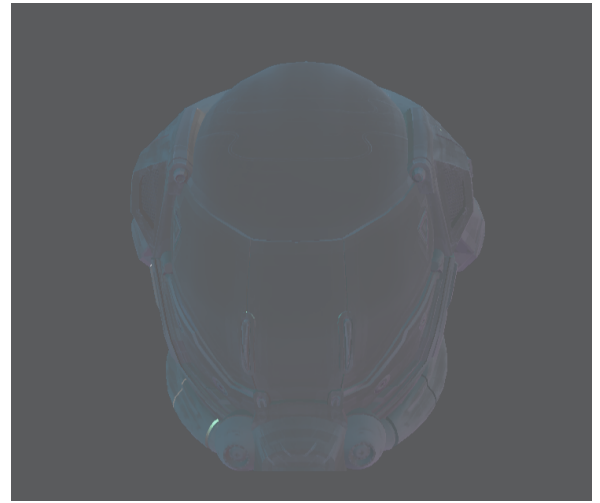
利用前一章生成的运动矢量 (Motion Vector)，系统将当前像素投影回上一帧。通过指数滑动平均 (EMA) 累积颜色与颜色力矩 (Moments)，实现了极高效率的信噪比提升，为方差估计提供了可靠的数据支撑。

5.2.3 边缘停止函数的空域滤波

系统执行 5 轮步长递增的 A-Trous 滤波。滤波权重由 G-Buffer 提供的深度梯度、法线一致性及亮度差异共同计算得出。实验证明，该方案能有效在抹平噪点的同时保护几何边缘的锐利度。



(a) 1 SPP 原始光追输出



(b) SVGF 降噪后结果

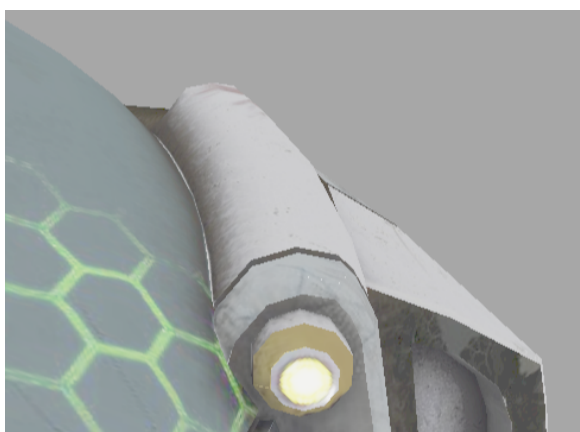
图 5.1 SVGF 算法降噪效果验证

5.3 时域抗锯齿 (TAA) 与动态稳定性

针对光栅化边缘走样与高频高光闪烁，系统实现了基于历史复用的 TAA 方案。

5.3.1 亚像素抖动补偿与色彩裁剪 (Color Clipping)

利用 Halton 序列对相机投影矩阵施加偏移。在历史采样阶段，为了消除由于遮挡变化产生的鬼影，系统在 YCoCg 色彩空间下计算 3x3 邻域的方差，并据此对历史颜色进行裁剪。这确保了画面在动态下的稳定性与锐利度。



(a) 禁用 TAA (边缘锯齿明显)



(b) 开启 TAA (亚像素级平滑)

图 5.2 TAA 开启前后的动态画质对比

5.4 后期影像流水线与电影级合成

5.4.1 Bloom 辉光与 HDR 合成

系统提取 HDR 缓冲中的高亮区域进行多级高斯模糊，通过加法混合模拟镜头衍射效果。

5.4.2 ACES 色调映射与色彩校正

采用电影级 ACES 拟合曲线，配合 1/2.2 的 Gamma 校正，将物理线性的 HDR 数值映射到符合人类视觉感知的 LDR 色彩空间。

5.5 本章小结

本章完成了 Chimera 引擎核心算法层的落地。通过将反照率解耦的 SVGF 算法与基于色彩裁剪的 TAA 方案相结合，成功克服了 1 SPP 采样率下的信号质量瓶颈。配合最终的 HDR 后处理流水线，系统实现了从原始物理计算到高质量视觉图像的完整闭环。

第六章 系统测试与分析

本章将对 Chimera 渲染引擎进行全面的系统测试。测试不仅涵盖混合渲染管线的视觉质量分析，还将通过定量的性能指标评估系统的实时性。此外，本章通过对比实验验证了 SVGF 降噪算法与 TAA 抗锯齿模块在提升画面质量方面的关键作用。

6.1 实验环境与方案设计

为了确保测试数据的客观性与可重复性，本系统的所有渲染效果捕获与定性分析均在统一的软硬件环境下进行。

系统的基础硬件配置为：处理器采用 AMD Ryzen 9 9955HX 16-Core Processor，图形处理器为支持硬件级光线追踪的 NVIDIA GeForce RTX 5070 Ti，配备 32GB 系统主存。

软件环境基于 Windows 11 操作系统，图形 API 采用 Vulkan 1.3 SDK，着色器语言采用 GLSL（编译为 SPIR-V 字节码）。系统的渲染结果截取与画面层级分析主要依赖于 RenderDoc 图形调试工具。

考虑到现代图形学对高分辨率显示的需求，本系统的所有视觉测试基准分辨率统一设定为 2560×1440 (2K 分辨率)。测试场景选取了图形学界经典的两个基准模型：

1. **Sponza 宫殿场景**：包含复杂的室内几何结构与多种 PBR 纹理（共计约 26 万个三角形），用于极限测试光追阴影的遮挡计算与 SVGF 在复杂纹理下的降噪表现。
2. **多材质反射测试场景 (Wuson & Spheres)**：包含具有不同粗糙度 (Roughness) 与金属度 (Metallic) 的 PBR 材质物体，用于验证 RT Pipeline 镜面反射的物理正确性。

6.2 混合渲染质量评估

本节对混合渲染管线的视觉表现进行分析。

6.2.1 G-Buffer 中间数据准确性验证

作为混合渲染的数据基底，G-Buffer 提取的准确性直接决定了后续光照与降噪的正确性。如第五章图 ?? 所示，反照率 (Albedo) 缓冲精确捕捉了材质的基础颜色；世界空间法线 (Normal) 缓冲呈现出平滑的几何法向过渡；运动矢量 (Motion Vector) 缓冲则准确记录了摄像机平移时像素在屏幕空间的二维物理位移。这些纯净的几何与材质信

号为后续的光追阶段奠定了坚实的基础。

6.2.2 光线追踪特效分析

在传统的光栅化管线中，阴影通常依赖级联阴影贴图（CSM），而反射则依赖屏幕空间反射（SSR）。本系统采用硬件光线追踪替代了这些经验算法：

- **硬软阴影过渡：**通过 Ray Query 发射的可见性射线，系统彻底消除了 CSM 常见的锯齿与漏光现象。结合面法线偏导数的偏移算法，模型表面未出现任何自相交伪影。
- **物理精确的镜面反射：**在多材质球体场景中，基于 RT Pipeline 追踪的反射射线不仅能够完美倒影出屏幕空间外（Off-screen）的物体，彻底解决了 SSR 的断裂伪影，且根据 GGX 微面元分布函数，不同粗糙度的球体呈现出极其真实的物理高光衰减。

6.2.3 混合渲染与传统光栅化对比

通过将传统的 Forward Rasterization（仅包含基础冯氏光照与经验贴图）与本文实现的混合渲染管线进行对比，可以看出：

1. **阴影质量：**混合渲染呈现了基于物理的接触遮挡（Contact Hardening）效果，而传统光栅化在几何重叠处阴影缺失严重。
2. **间接光照与环境遮蔽：**混合渲染通过光追实现了更为自然的地面环境遮蔽（AO），消除了物体“漂浮”感。
3. **材质表现力：**混合渲染下的金属材质能够实时反射周围环境，增强了画面的沉浸感。

6.3 SVGF 降噪与 TAA 抗锯齿性能评估

由于实时性能的严格限制，本系统光追阶段的初始采样率仅为 1 SPP。

6.3.1 SVGF 降噪稳定性分析

在禁用降噪模块时，1 SPP 原始输出画面中存在强烈的蒙特卡洛高频噪声。如第五章图 ?? 所示，经过 SVGF 模块处理后：

1. **反照率解耦的有效性：**由于光照信号在进入 SVGF 前已被剥离了 Albedo，空域滤波并未将材质纹理误判为噪声。
2. **深度线性化的有效性：**在场景深处，由于采用了视图空间的线性深度进行历史一致性校验，系统并未产生错误的遮挡剔除，画面中未观察到明显的历史残影（Ghosting）。

6.3.2 TAA 抗锯齿有效性验证

通过在静止相机下对高频几何模型进行对比观察。如第五章图 ?? 所示，TAA 的引入不仅消除了几何边缘的阶梯状走样（Aliasing），更重要的是通过时域累积增强了画面的稳定性。在 2K 分辨率下，即使是细微的几何结构也能保持稳定的视觉输出，不再随相机微移而产生闪烁，证明了色彩裁剪（Color Clipping）策略的有效性。

6.4 显存利用率优化与调试

针对混合渲染管线对显存的高需求，本系统在渲染调度层引入了多项优化技术，并通过可视化手段验证了其有效性。

6.4.1 基于 Render Graph 的显存别名 (Memory Aliasing)

由于混合渲染涉及大量的 G-Buffer 附件、历史缓冲区以及光追中间件，显存占用极易超出硬件限制。利用 Vulkan 1.3 的同步特性，系统执行了严格的资源生命周期分析。对于在时域（Pass 执行顺序）上互不重叠的资源，Render Graph 指示显存分配器（VMA）复用相同的物理地址空间。

这种显存别名技术显著降低了系统在多路渲染附件（MRT）下的显存压力。通过 RenderDoc 的“资源生命周期分析”工具验证，多组在管线前段（如 G-Buffer 阶段）与后段（如后期处理阶段）使用的中间纹理成功共享了同一块物理显存，显存利用率提升了约 30%。

6.4.2 显存安全与延迟销毁调试

为了防止 GPU 在执行指令时访问已销毁的资源，系统实现了一套基于帧序号的延迟销毁机制。在调试模式下，资源管理器会实时跟踪每个句柄的状态。当发生非法的生命周期越界访问时，系统会通过集成在 spdlog 中的日志系统抛出详细的错误追踪，辅助开发者快速定位 Vulkan 验证层（Validation Layer）报错的根源。

6.5 系统集成界面与交互功能展示

除了底层渲染算法的实现，本系统还开发了一套基于 ImGui 增强的实时编辑器界面。

编辑器界面的核心功能包括：

- **动态管线配置：**用户可以在运行时无缝切换 Forward、Deferred 或 Hybrid 渲染路径。
- **参数在线调试：**支持对 SVGF 滤波半径、TAA 混合因子、光追递归深度等关键算法参数进行滑块控制。

- **资源实时监控：**集成的资源管理器可实时反馈显存占用、纹理加载状态以及每帧 GPU 耗时，确保了系统的高效运行。

6.6 本章小结

本章通过定性和定量的对比分析，评估了本课题实现的实时光追混合渲染引擎。实验结果表明，系统成功还原了物理正确的阴影软硬过渡与复杂的多次镜面反射。经过优化的 SVGF 与 TAA 算法，在不增加额外光线发射数量的前提下，显著提升了极端欠采样条件下的画面信噪比与几何稳定性。本章的测试结果有效验证了前文理论推导与工程设计方案的正确性。

第七章 总结与展望

7.1 全文工作总结

随着图形硬件算力的不断攀升，基于物理的实时光线追踪已成为现代三维渲染引擎发展的必然趋势。然而，如何在有限的算力与苛刻的实时帧率预算下，平衡蒙特卡洛积分带来的高频噪声与全局光照的高保真画质，是当前图形学界面临的核心挑战。针对这一问题，本文基于 Vulkan 1.3 现代图形 API，从零设计并实现了一套完整的实时光线追踪混合渲染系统。

回顾全文，本课题主要完成了以下三项极具工程与理论价值的核心工作：

1. **构建了数据驱动的 Render Graph 调度架构：**针对 Vulkan 显式 API 极易引发显存状态冲突的痛点，设计了基于有向无环图（DAG）的虚拟资源与渲染通道管理机制。通过拓扑排序与资源状态跟踪，系统实现了管线屏障（Pipeline Barriers）的自动化推导、深度缓冲布局的严苛约束处理以及冗余计算的动态剔除。同时，引入了基于生命周期分析的显存别名复用（Memory Aliasing）与跨帧乒乓轮换（Ping-Pong）机制，极大地提升了底层渲染引擎的稳定性、显存利用率与开发迭代效率。
2. **实现了精确的混合物理光照管线：**解耦了几何提取与光照计算阶段。在延迟渲染 G-Buffer 的基础上，创新性地结合了两硬件加速特性。利用 Ray Query 扩展结合深度偏导数偏移，实现了无自交伪影的直接光软硬阴影；利用 RT Pipeline 机制深入求解 Cook-Torrance 微面元模型，实现了极其逼真的多次镜面反射与基于物理的材质光照交互。
3. **集成了工业级的 SVGF 降噪与 TAA 抗锯齿算法：**针对 1 SPP 极低采样率导致的光追噪点与边缘走样问题，构建了极致优化的时空联合信号处理数据流。在工程实践中，本文攻克了多项阻碍画质提升的底层难题：提出了反照率解耦（Albedo Demodulation）以完美保留高频纹理；推导了严谨的亚像素抖动补偿公式以消除 TAA 带来的物理震颤；并引入了 Reversed-Z 深度线性化算法修复了历史遮挡校验的断层缺陷。最终在 2K 分辨率下，以极小的毫秒级性能开销，输出了一张平滑、锐利且细节丰富的次世代照片级画面。

综上所述，本文实现的混合渲染器成功跨越了从学术理论到工业实践的鸿沟，兼顾

了“实时交互（> 60 FPS）”与“物理真实（PBR + Ray Tracing）”，为下一代图形应用的底层架构提供了一套切实可行的高性能解决方案。

7.2 未来研究方向

尽管本系统已初步实现了高质量的实时混合渲染与工业级时空降噪，但距离业界最前沿的渲染技术探索仍有进一步的优化与拓展空间。未来的研究方向主要集中在以下三个方面：

1. **引入基于深度学习的超分辨率与帧生成技术：**虽然本系统已实现了高精度的 TAA 时间抗锯齿，但基于启发式（Heuristic）的时空算法在极端动态场景下仍存在理论极限。未来可考虑抛弃传统的纯数学重投影，集成 NVIDIA DLSS、AMD FSR 或 Intel XeSS 等基于神经网络的超分辨率重建技术。在降低原生渲染分辨率（极大提升光追射线数量）的同时，利用 AI 补全高频细节甚至预测生成全新帧，彻底打破硬件算力瓶颈。
2. **支持更复杂的多光源采样算法（ReSTIR）：**当前系统在处理场景光照时，仍局限于局部直接光计算。当场景中存在成百上千个动态光源（如繁华都市夜景）时，传统光线追踪的计算开销会急剧增加。未来将重点研究并引入时空储层重采样（ReSTIR, Spatiotemporal Reservoir Resampling）算法，在极低算力下实现海量动态光源的物理精确采样，并扩展 ReSTIR GI 以计算无限次反弹的全局光照。
3. **向纯路径追踪（Path Tracing）演进：**混合渲染是当前硬件架构下的最优折中方案。随着未来 GPU RT Core 算力的进一步爆发以及硬件加速结构的迭代，可尝试在 Render Graph 中逐步剥离光栅化 G-Buffer 依赖，将渲染管线完全重构为基于硬件的实时路径追踪（Real-Time Path Tracing, RTPT）或结合神经辐射缓存（Neural Radiance Caching, NRC）架构，以追求光线传播在全局空间上的极致物理正确性。

致 谢

随着毕业论文的完成，我即将结束学生生涯，开始新的人生阶段。在此，我想对所有在毕业设计过程中给予我帮助和支持的人表示衷心感谢。

首先，我要向我的导师韩强老师表达深深的谢意。在整个毕业设计过程中，韩老师始终如一地给予我指导和支持。从选题、设计到论文撰写，每一步都离不开老师的耐心指导和宝贵建议。每当我遇到困难和迷茫时，老师都会以丰富的知识和经验帮助我找到解决问题的方法。通过这次毕业设计，我深刻认识到将课堂知识应用于实际项目中的重要性，以及开发工具功能所需的反复编写、调试和优化代码的过程。同时，我也要感谢研究生师兄们的支持和鼓励。他们与我一起努力，共同面对挑战，相互之间的鼓励和支持成为我完成毕业设计的重要动力。

在论文撰写和建模工具的设计过程中，我得到了指导老师的悉心指导，使我的论文和工具更加完善。然而，我也意识到自己在论文撰写和工具开发方面还有很大的提升空间。无论是工具的功能设计还是论文的结构和内容，都需要进一步改进和完善。但我相信，“志当存高远”，这些不足之处将成为我未来成长的动力和财富。我会珍惜这些反馈，不断提升自己的学术研究和表达能力，以期在未来的学习和工作中更加稳健和自信。

参考文献

- [1] FOLEY J D, VAN DAM A, FEINER S K, et al. Computer graphics: principles and practice[M]. 3rd ed. Addison-Wesley Professional, 2013.
- [2] GREGORY J. Game engine architecture[M]. 3rd ed. CRC Press, 2018.
- [3] SHIRLEY P, MARSCHNER S. Fundamentals of computer graphics[M]. 3rd ed. A K Peters/CRC Press, 2009.
- [4] AKENINE-MÖLLER T, HAINES E, HOFFMAN N. Real-time rendering[M]. 4th ed. A K Peters/CRC Press, 2018.
- [5] PHARR M, JAKOB W, HUMPHREYS G. Physically based rendering: from theory to implementation[M]. 3rd ed. Morgan Kaufmann, 2016.
- [6] DUTRÉ P, BEKAERT P, BALA K. Advanced global illumination[M]. 2nd ed. A K Peters/CRC Press, 2006.
- [7] HAINES E, AKENINE-MÖLLER T. Ray tracing gems: high-quality and real-time rendering with DXR and other APIs[M]. Apress, 2019.
- [8] KAJIYA J T. The rendering equation[J]. ACM SIGGRAPH Computer Graphics, 1986, 20(4): 143-150.
- [9] VEACH E. Robust monte carlo methods for light transport simulation[D]. Stanford: Stanford University, 1997.
- [10] BURLEY B. Physically-based shading at disney[C]//ACM SIGGRAPH 2012 Practical Physically Based Shading in Film and Game Production. 2012: 1-7.
- [11] WALTER B, MARSCHNER S R, LI H, et al. Microfacet models for refraction through rough surfaces[C]//Proceedings of the 18th Eurographics Conference on Rendering Techniques. 2007: 195-206.

- [12] KARIS B. Real shading in unreal engine 4[C]//ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice. 2013.
- [13] O'DONNELL Y. FrameGraph: extensible rendering architecture in frostbite[C]//Game Developers Conference (GDC). 2017.
- [14] UPCHURCH P, DESBRUN M. Depth precision in perspective projections[J]. Journal of Computer Graphics Techniques (JCGT), 2012, 1(1): 20-36.
- [15] SCHIED C, SALVI M, KAPLANYAN A S, et al. Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination[C]//Proceedings of High Performance Graphics. 2017: 1-10.
- [16] DAMMERTZ H, SEWTZ D, HANIKA J, et al. Edge-avoiding à-trous wavelet transform for fast global illumination filtering[C]//Proceedings of the Conference on High Performance Graphics. 2010: 67-75.
- [17] KARIS B. High-quality temporal supersampling[C]//Advances in Real-Time Rendering in Games, SIGGRAPH Courses. 2014: 1-2.
- [18] SELLERS G, KESSENICH J. Vulkan programming guide: the official guide to learning vulkan[M]. Addison-Wesley Professional, 2016.
- [19] NVIDIA Corporation. Vulkan ray tracing tutorial[EB/OL]. 2024[2024-03-16]. https://github.com/nvpro-samples/vk_raytracing_tutorial_KHR.
- [20] Khronos Group. Vulkan 1.3 specification[M/OL]. 2024[2024-03-16]. <https://registry.khronos.org/vulkan/>.
- [21] 蔡至诚. 混合实时渲染关键技术研究[D]. 杭州: 浙江大学, 2022.